
boolean



React Router

Single Page Application e Routing

SPA e Routing

Intro

Le Single Page Applications (**SPA**) sono applicazioni web che caricano una sola pagina HTML iniziale dal server.

Successivamente, il contenuto viene aggiornato dinamicamente tramite JavaScript (**Client Side Rendering**) senza la necessità di ricaricare l'intera pagina durante l'interazione dell'utente.

In alcuni casi, i dati possono essere recuperati da web **API**, permettendo un'esperienza utente più reattiva e dinamica.

Gestire la visualizzazione di diverse sezioni (pagine) di un'applicazione in base a determinate casistiche può risultare un lavoro molto complesso.
Ecco perché sistemi di routing come **React Router** ci aiutano a **separare le diverse sezioni dell'applicativo in base all'URL** che l'utente intende visualizzare.

SPA e Routing

Esempio pratico! 😎

Immaginiamo di dover sviluppare un'applicazione che consenta agli utenti di visualizzare diverse pagine:

- una pagina **home**
- una pagina con un **elenco di prodotti**
- una pagina per **contattarci**
- e così via...

Senza un sistema di routing saremmo costretti ad aggiungere diverse variabili di stato e condizioni complesse per ottenere il comportamento desiderato.

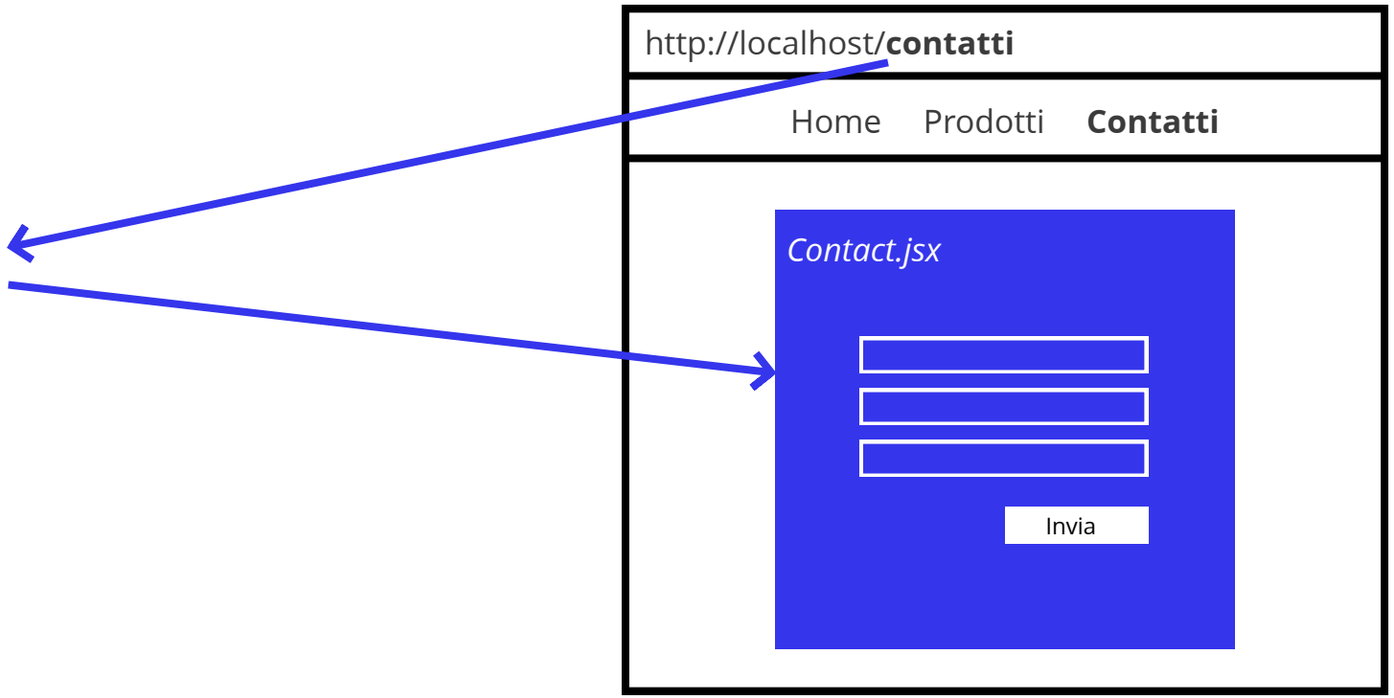
Questo approccio renderebbe il nostro codice difficile da gestire e complicato da mantenere.

React Router

La libreria più utilizzata di React! 😎

React Router ci permette di **mappare le rotte** della nostra applicazione e, in base all'**url**, mostrare un **componente diverso**!

- rotta **/** **Home.jsx**
- rotta **/prodotti** **Products.jsx**
- rotta **/contatti** **Contact.jsx**
- ecc...



React Router

Installazione e configurazione

Installiamo l'ultima versione della libreria utilizzando **pnpm.**

```
pnpm add react-router
```

Nel file **App.jsx**, integriamo il sistema di routing nell'applicazione importando il componente **BrowserRouter** da React Router.

Questa configurazione ci permetterà di **definire e controllare tutte le rotte esistenti nella nostra applicazione**

```jsx
import { BrowserRouter } from "react-router";

export default function App() {
  return <BrowserRouter></BrowserRouter>;
}
```

React Router

Mappatura rotte

Ora possiamo iniziare a definire le rotte della nostra applicazione!

Utilizziamo il componente **Routes** per raggruppare tutte le nostre rotte.

Inseriamo poi un componente **Route** per ogni singola rotta che vogliamo gestire!

```
1  import { BrowserRouter, Routes, Route } from "react-router";
2  // Pages
3  import HomePage from "./pages/HomePage";
4
5  export default function App() {
6    return (
7      <BrowserRouter>
8        <Routes>
9          <Route path="/" element={<HomePage />} />
10       </Routes>
11     </BrowserRouter>
12   );
13 }
```

React Router

Mappatura rotte

All'interno del componente **Route** abbiamo due props importanti:

path ci permette di definire quale sarà l'indirizzo della nostra rotta (es: */products*)

element ci permette di inserire il Componente che sarà usato come pagina quando il nostro url corrisponde al path!

```
1  import { BrowserRouter, Routes, Route } from "react-router";
2  // Pages
3  import HomePage from "./pages/HomePage";
4
5  export default function App() {
6    return (
7      <BrowserRouter>
8        <Routes>
9          <Route path="/" element={<HomePage />} />
10       </Routes>
11     </BrowserRouter>
12   );
13 }
```

React Router

Mappatura rotte

Al posto della prop **element** possiamo anche utilizzare la prop **Component**

element si aspetta l'intero tag del componente.

Component ci permette di inserire soltanto il nome del componente

```
1  import { BrowserRouter, Routes, Route } from "react-router";
2  // Pages
3  import HomePage from "./pages/HomePage";
4
5  export default function App() {
6    return (
7      <BrowserRouter>
8        <Routes>
9          <Route path="/" Component={HomePage} />
10       </Routes>
11     </BrowserRouter>
12   );
13 }
```



LIVE CODING

Prime Pagine

React Router

Come linkare le pagine

Per navigare le pagine dell'applicazione, React Router mette a disposizione il componente **Link**.

Possiamo utilizzare **<Link>** in qualsiasi parte dell'applicazione per definire i collegamenti tra le pagine. Ad esempio:

<Link to="/">Home Page</Link>

In aggiunta al componente **Link**, abbiamo a disposizione il componente **NavLink** che aggiunge automaticamente una classe "active" quando la rotta corrente corrisponde a quella specificata nel link.

```
1  import { Link, NavLink } from "react-router";
2
3  export default function MainNav() {
4    return (
5      <nav>
6        <Link to="/">Home Page</Link>
7        <NavLink to="/prodotti">Prodotti</NavLink>
8      </nav>
9    );
10 }
```



LIVE CODING

Creiamo il menù di navigazione

React Router

Come definire Layouts

Abbiamo creato e collegato diverse pagine rendendo effettivamente la nostra applicazione una **SPA** 😎.

Tuttavia, pensate che sia efficiente replicare per ogni pagina l'header con il menu di navigazione e il footer? 🤔

React Router ci offre la possibilità di creare **rotte annidate**.

In questo modo, possiamo sfruttare una **rotta genitore** che visualizzi un **layout con i componenti comuni** tra le pagine (ad esempio, l'header e il footer).

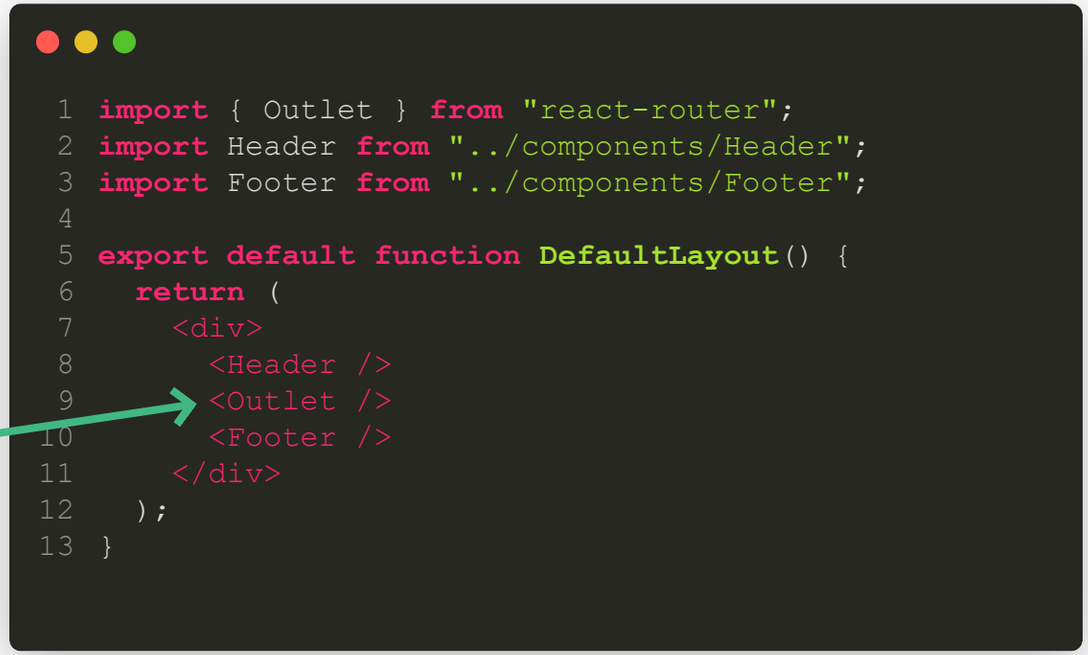
All'interno di questa, avremo le **rotte figlie** che mostreranno il **contenuto specifico delle pagine**.

React Router

Creiamo il primo Layout

Iniziamo con la creazione di un layout di base chiamato **DefaultLayout**, il quale includerà l'Header, il Footer e il componente speciale **<Outlet />**.

<Outlet /> è un componente di React Router che possiamo immaginare come un placeholder in cui sarà visualizzato il **componente di pagina associato alla rotta figlia corrente**.
È simile alla prop children che abbiamo studiato nelle scorse lezioni.



```
1  import { Outlet } from "react-router";
2  import Header from "../components/Header";
3  import Footer from "../components/Footer";
4
5  export default function DefaultLayout() {
6    return (
7      <div>
8        <Header />
9        <Outlet />
10       <Footer />
11     </div>
12   );
13 }
```

React Router

Utilizziamo il Layout

Passiamo ora all'utilizzo del Layout.

Definiamo una **rotta genitore**, il cui elemento sarà il componente **DefaultLayout** precedentemente importato.

All'interno di questo componente genitore, definiamo le **rotte figlie** che, a seconda della rotta, mostreranno il componente impostato come **element al posto del placeholder <Outlet />**.

```jsx
import { BrowserRouter, Routes, Route } from "react-router";
// Layout
import DefaultLayout from "./layouts/DefaultLayout";
// Pages
import HomePage from "./pages/HomePage";
import Products from "./pages/Products";
import Contact from "./pages/Contact";

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route element={<DefaultLayout />}>
          <Route path="/" element={<HomePage />} />
          <Route path="/prodotti" element={<Products />} />
          <Route path="/contatti" element={<Contact />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```



LIVE CODING

Creiamo un layout di base

React Router

Mappatura rotte dinamiche

In alcuni casi può essere utile creare rotte che accettino **parametri dinamici**.

Ad esempio, possiamo definire un unico componente di pagina per la visualizzazione di un singolo prodotto e, in base all'ID passato come parametro nell'URL, mostrare i dettagli corrispondenti.

Per ottenere questo comportamento, aggiungiamo il **nome del parametro preceduto dai due punti** nel nostro percorso (path), come ad esempio: path="/prodotti/**:id**".

```jsx
import { BrowserRouter, Routes, Route } from "react-router";
// Layout
import DefaultLayout from "./layouts/DefaultLayout";
// Pages
import HomePage from "./pages/HomePage";
import Products from "./pages/Products";
import Contact from "./pages/Contact";
import SingleProduct from "./pages/SingleProduct";

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route element={<DefaultLayout />}>
          <Route index element={<HomePage />} />
          <Route path="/prodotti" element={<Products />} />
          <Route path="/contatti" element={<Contact />} />
          <Route path="/prodotti/:id" element={<SingleProduct />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

React Router

Leggere i parametri dalla rotta

Per accedere alle informazioni provenienti da rotte dinamiche, il componente associato può utilizzare l'hook **useParams** messo a disposizione da React Router.

L'hook **useParams** ci consente di **estrarre** questi **parametri dalla URL** e utilizzarli per vari scopi.
Ad esempio, possiamo effettuare chiamate API al caricamento del componente, visualizzarli direttamente all'interno del JSX, oppure utilizzarli per definire il comportamento e il rendering del componente in modo dinamico in base ai dati presenti nella URL.

```jsx
import { useEffect } from "react";
import { useParams } from "react-router";

export default function SingleProduct() {
  const { id } = useParams();

  useEffect(() => {
    // fetch dati usando l'id
  }, []);

  return (
    <div>
      <h1>ID prodotto: {id}</h1>
    </div>
  );
}
```

React Router

Navigazione programmatica

La navigazione programmatica si riferisce all'azione di spostare l'utente su un'altra pagina senza che l'utente effettui un click su un link.
È particolarmente utile quando vogliamo reindirizzare l'utente a una pagina specifica dopo aver inviato un form oppure quando una risorsa con parametro dinamico non è stata trovata.

React Router ci mette a disposizione l'hook **useNavigate**. Questo hook ci permette di navigare in modo programmatico tra le pagine dell'applicazione.

```
1  import { useEffect } from "react";
2  import { useNavigate, useParams } from "react-router";
3
4  export default function SingleProduct() {
5    const { id } = useParams();
6    const navigate = useNavigate();👈
7
8    useEffect(() => {
9      async function fetchProduct() {
10       try {
11         const response = await fetch(`https://fakestoreapi
12         if (!response.ok) {
13           if(response.status === 404){
14             // redirect verso archivio prodotti
15             navigate('/products');
16           } else {
17             throw new Error('Error fetching product data, f
18           }
19         }
20
21         // ...
22
23       } catch(e) {
24         console.log(e.message)
25         // Logica feedback errore in UI
26       }
27     }
28
29     fetchProduct();
30
31   }, [id, navigate]);
32
33   ...
34 }
```

ESERCIZIO